

VoiceLoop — dokumentacja architektury i handoff dla kolejnego AI

Wersja projektu: 0.2.0

Platforma: Windows, uruchomienie lokalne

Stan dokumentu: 11 sierpnia 2026 **Repozytorium:** C:\Users\marci\VoiceLoop

1. Po co istnieje ten dokument

Ten plik jest jednocześnie:

1. szczegółowym README technicznym,
2. mapą architektury i odpowiedzialności modułów,
3. instrukcją uruchamiania i diagnostyki,
4. opisem granic bezpieczeństwa i prywatności,
5. handoffem, który można przekazać kolejnemu agentowi AI,
6. listą ograniczeń oraz logicznych następnych kroków.

Dokument opisuje **rzeczywisty stan kodu**, a nie wyłącznie wizję produktu. Nie zawiera wartości kluczy API, tokenów ani klucza licencji VoiceAttack.

2. Prompt startowy dla kolejnego chatu lub agenta

Poniższy blok można wkleić jako pierwszą wiadomość do innego AI:

Kontynuujesz projekt VoiceLoop znajdujący się w:

C:\Users\marci\VoiceLoop

VoiceLoop to lokalny asystent Windows z bezpiecznym rdzeniem FastAPI.

Gemini jest głównym mózgiem rozmowy. Venice pozostaje opcjonalnym providerem chmurowym, a zadania przechodzą przez typowany, walidowany planer.

LM Studio pełni dwie lokalne role:

- 1) Qwen 2.5 14B Instruct jest fallbackiem czatu,
- 2) Nomic Embed generuje lokalne embeddingi.

Screenpipe rejestruje lokalny kontekst aktywności. Worker VoiceLoop tworzy embeddingi metadanych aktywności i zapisuje je w SQLite. Przy nowym zapytaniu retriever wybiera podobne wpisy i przekazuje kilka trafnych fragmentów do aktywnego planera jako kontekst.

n8n obsługuje prosty routing dokładnych intencji. UI.Vision wykonuje dozwolone makra ekranowe. VoiceAttack rozpoznaje stałe komendy i frazę „Asystent” otwiera jednorazowy nasłuch swobodnej wypowiedzi. Deepgram wykonuje polskie STT oraz selektywną transkrypcję zakończonych spotkań wykrytych przez Screenpipe.

Najważniejsze reguły:

- nie ujawniaj ani nie commituj listener/.env, data/, logów i tokenów;
- LLM może zwracać tylko dozwolone action_id i argumenty;
- nie dodawaj dowolnego wykonywania shell z odpowiedzi modelu;
- nie obniżaj poziomu ryzyka istniejących akcji;
- operacje high-risk zawsze wymagają potwierdzenia;
- zachowaj STOP, deduplikację i pojedynczą kolejkę wykonawczą;
- przed zmianami przeczytaj docs/VOICELOOP_ARCHITECTURE_HANDOFF.md;
- po zmianach uruchom ruff, pytest i scripts/test-loop.ps1;
- panel odczytuje `LLM\_PRIMARY` z sesji; przy trybie cloud informuje, że Venice jest modelem głównym, a checkbox dotyczy tylko fallbacku w trybie local-first.

Najpierw sprawdź:

GET http://127.0.0.1:8765/api/v1/health

Następnie opisz, które pliki zamierzasz zmienić i dlaczego.

3. Cel produktu

VoiceLoop ma być osobistym asystentem Windows, który:

- rozumie polecenia po polsku,

- przyjmuje wejście z panelu, mikrofonu, VoiceAttack lub lokalnego API,
- pamięta fakty i preferencje użytkownika,
- korzysta z historii lokalnej aktywności,
- potrafi planować działania wieloetapowe,
- automatyzuje Windows oraz UI przez dozwolone akcje,
- pyta o potwierdzenie przed operacjami ryzykownymi,
- daje się natychmiast zatrzymać,
- zachowuje możliwie dużo danych lokalnie,
- używa chmurowego modelu do rozumowania, ale nie daje mu dowolnego dostępu do systemu.

Projekt nie jest ogólnym agentem shell. Jego główną granicą bezpieczeństwa jest lokalna allowlista akcji oraz walidowany format planu.

4. Aktualny stan operacyjny

Stan zweryfikowany na żywym systemie:

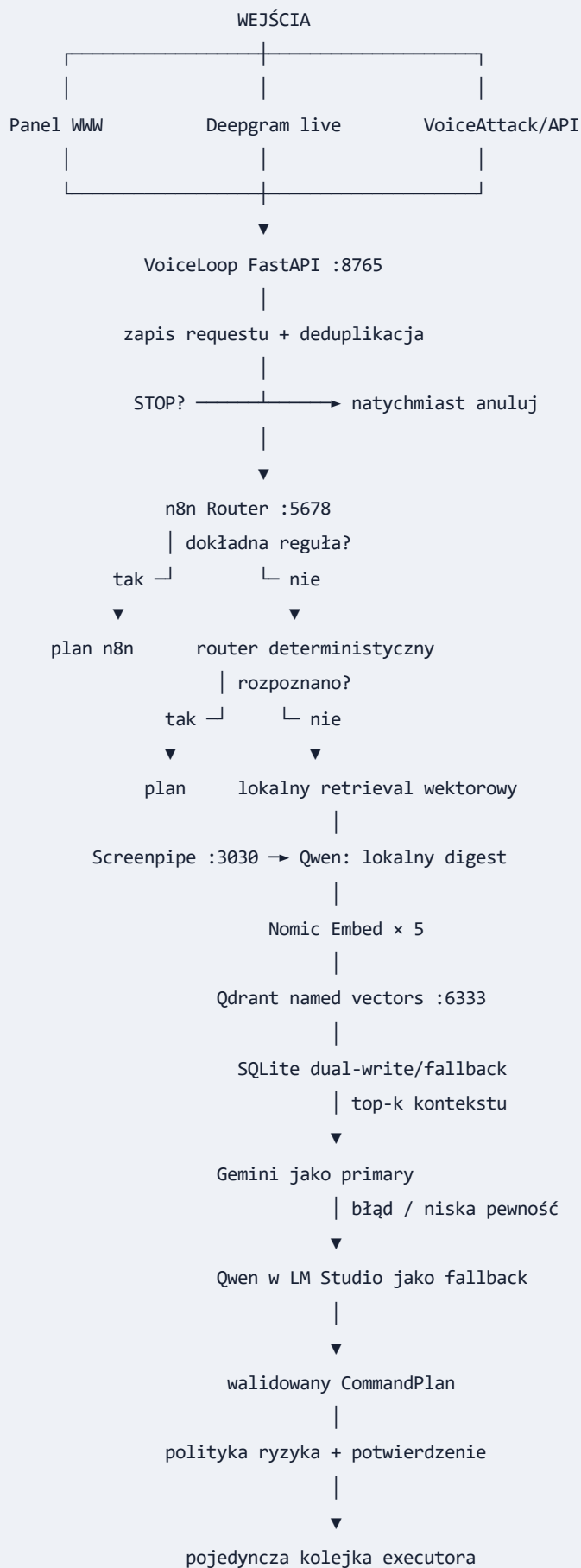
Komponent	Stan	Rola
VoiceLoop FastAPI :8765	OK	rdzeń, API, panel, kolejka
Gemini	OK	główny model rozmowy
Venice AI	OK	opcjonalny provider chmurowy
LM Studio Qwen 2.5 14B	OK	fallback czatu i lokalna analiza aktywności
LM Studio Nomic Embed	OK	lokalne embeddingi, 768 wymiarów
Qdrant :6333	OK	pięć przestrzeni wektorowych pamięci lokalnej
n8n :5678	wyłączony	opcjonalny router prostych intencji
Screenpipe :3030	OK	lokalne źródło aktywności i spotkań
Screenpipe vector worker	OK	Qwen → Nomic → Qdrant + dual-write SQLite
Screenpipe meeting worker	OK	oczekiwanie na zakończone spotkania
UI.Vision	OK	dozwolone automatyzacje ekranowe
VoiceAttack	OK	profil v2: 15 stałych komend i wyzwalanie one-shot Deepgram
Deepgram live listener	OK	Nova-3 PL, one-shot, streaming diarization

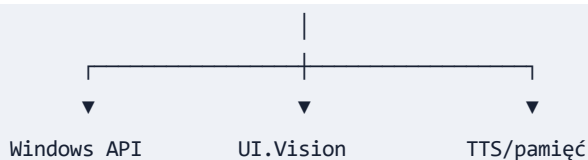
Aktualne modele:

- główny model rozmowy: `gemini-3.5-flash` ,
- opcjonalny provider: `venice-uncensored-1-2` ,
- lokalny fallback: `qwen2.5-14b-instruct-1m-abliterated` ,
- embeddingi: `text-embedding-nomic-embed-text-v2-moe` .

Po ostatnich zmianach pełny zestaw 200 testów i Ruff przechodzą bez błędów.

5. Architektura wysokiego poziomu





6. Porty i procesy

Port	Usługa	Dostęp	Przeznaczenie
1234	LM Studio	127.0.0.1	chat fallback i embeddings
3030	Screenpipe	127.0.0.1	lokalne API historii aktywności
6333	Qdrant	127.0.0.1	lokalna pamięć named vectors
5678	n8n	127.0.0.1	edytor, health i webhook
5679	n8n task broker	127.0.0.1	proces pomocniczy n8n
8765	VoiceLoop	127.0.0.1	panel, REST API i SSE

Usługi mają działać wyłącznie na loopback. Projekt nie jest przygotowany do bezpośredniego wystawienia tych portów do sieci LAN ani Internetu.

7. Pełny przebieg polecenia

7.1. Przyjęcie wejścia

Każde polecenie jest normalizowane do `CommandRequest` :

- `schema_version` ,
- unikalny `request_id` ,
- `source` ,
- `text` albo `command_id` ,
- `include_screen` ,
- `allow_cloud` ,
- znacznik czasu.

Źródła:

- `panel` ,
- `deepgram` ,
- `voiceattack` ,
- `api` ,
- `n8n` .

7.2. Zapis i deduplikacja

`AssistantService` :

1. tworzy fingerprint z unormowanego tekstu,
2. sprawdza krótkie okno deduplikacji,
3. zapisuje komendę w SQLite,
4. dodaje wiadomość użytkownika do historii,
5. ustawia status `planning` ,
6. publikuje zdarzenie SSE.

Powtórzenie identycznej komendy w oknie deduplikacji zwraca istniejący `request_id` zamiast wykonywać zadanie drugi raz.

7.3. Priorytet STOP

STOP jest rozpoznawany przed zwykłym planowaniem. Powoduje:

- zatrzymanie Deepgram live,
- anulowanie oczekujących potwierdzeń,
- opróżnienie kolejki,
- anulowanie bieżącego wykonania,
- zatrzymanie TTS,
- zakończenie bieżącego procesu UI.Vision,
- publikację zdarzenia `stop` .

7.4. Potwierdzana przerwa czasowa

`VoiceConversationCoordinator` rozpoznaje lokalnie frazę:

Przerwij działanie na <liczba> <sekund|minut|godzin>

Komenda nie trafia do modelu ani executora. Koordynator zapisuje oczekującą wartość i zawsze pyta o potwierdzenie. Dopiero `potwierdzam` przełącza stan na `paused` ; `anuluj` usuwa oczekującą przerwę. W stanie `paused` Deepgram pozostaje aktywny wyłącznie po to, aby rozpoznać `wznów działanie / włącz się / koniec przerwy` ; wszystkie inne transkrypty są ignorowane i nie trafiają do LLM ani akcji. Osobny task wznawia sesję automatycznie po upływie czasu. Limit jednej przerwy wynosi 24 godziny. Parser przyjmuje cyfry i polskie liczebniki 1–99. Callback aplikacji przekazuje tę komendę do koordynatora również poza aktywną sesją `managed conversation`, dlatego zawsze przechodzi ona przez potwierdzenie.

7.5. Routing n8n

Pierwszą próbą dla zwykłej komendy jest webhook n8n:

```
POST http://127.0.0.1:5678/webhook/voice-command-v1
```

Workflow `n8n/voice-loop.json` rozpoznaje dokładne, proste intencje:

- test pętli,
- otwarcie kalendarza,
- otwarcie przeglądarki,
- otwarcie czatu.

Jeśli n8n odpowie `unknown`, `none` albo `no_action`, VoiceLoop przechodzi dalej. Jeśli n8n nie działa, błąd jest traktowany jako fallback, a nie awaria całego asystenta.

7.6. Router deterministyczny

`router.py` obsługuje bez modelu:

- `voice_test`,
- `open_calendar`,
- `open_browser`,
- `open_chat`,
- opis aktywnego okna,
- opis ostatniej aktywności Screenpipe,
- STOP,
- proste tworzenie notatki.

Ta ścieżka jest szybka, tania i przewidywalna.

7.7. Retrieval lokalnej pamięci wektorowej

Jeśli komenda wymaga modelu:

1. aktualne zapytanie jest wysyłane do lokalnego endpointu embeddings,
2. powstaje wektor 768-wymiarowy,
3. Qdrant odpytuje pięć przestrzeni: `semantic`, `topic`, `intent`, `decision` i `person_context`,
4. wyniki są łączone wagami `0.40/0.20/0.15/0.15/0.10`,
5. do pamięci kontekstowej dodawanych jest maksymalnie `VECTOR_MEMORY_CONTEXT_LIMIT` fragmentów,
6. fragmenty są przekazywane do głównego planera.

Jeśli Qdrant jest wyłączony, niedostępny albo nie zwróci trafień, asystent korzysta z wcześniejszego wyszukiwania cosine w SQLite. SQLite pozostaje więc lokalnym fallbackiem i celem opcjonalnego dual-write.

7.8. Venice jako opcjonalny planer chmurowy

Przy `LLM_PRIMARY=cloud` lub `LLM_PRIMARY=venice`:

- Venice otrzymuje aktualne polecenie,
- dostaje definicje dozwolonych akcji,
- dostaje do 12 ostatnich wiadomości rozmowy,
- dostaje pamięć jawną i trafne fragmenty vector memory,
- przy `include_screen=true` może otrzymać metadane aktywnego okna oraz obraz,
- zwraca ustrukturyzowany JSON zgodny ze schematem `ProposedPlan`.

Model nie może zwrócić dowolnej komendy systemowej. Kroki o nieznanym `action_id` są odrzucane podczas konwersji planu.

7.9. Qwen jako lokalny fallback

Jeśli Venice:

- jest niedostępne,
- zwróci błąd,
- albo zwróci plan o zbyt niskiej pewności,

router może użyć Qwen z LM Studio. Fallback dostaje pustą historię, pustą pamięć i bez obrazu ekranu. Ogranicza to ilość prywatnego kontekstu przekazywanego do ścieżki awaryjnej i upraszcza zachowanie fallbacku.

7.10. Egzekucja planu

Przed kolejną każdy krok przechodzi przez `ActionRegistry.enforce_policy()`:

- nieznane akcje są odrzucane,
- model nie może obniżyć ryzyka zadeklarowanego lokalnie,
- lokalny wymóg potwierdzenia ma pierwszeństwo,
- każdy krok `high` zawsze wymaga potwierdzenia.

Executor:

- ma ograniczoną kolejkę,
 - wykonuje tylko jeden plan naraz,
 - respektuje zależności między krokami,
 - kończy plan po pierwszym błędzie akcji,
 - zapisuje wyniki i czasy wykonania,
 - publikuje zdarzenia do panelu.
-

8. Modele i podział odpowiedzialności

8.1. Gemini — główny model rozmowy

Odpowiada za:

- rozumienie wolnego języka,
- odpowiedzi konwersacyjne,
- wybór intencji,
- dobór akcji z allowlisty,
- tworzenie planów wieloetapowych,
- decyzję o potrzebie doprecyzowania,
- ocenę pewności i proponowanego ryzyka.

Rozmowa używa zwykłego tekstu z kontrolą `finish_reason`, długości i końcowej interpunkcji. Zadania używają osobnego kontraktu JSON Schema. Gemini **nie wykonuje akcji bezpośrednio**; lokalny kod filtruje i waliduje plan.

8.2. Venice AI — opcjonalny provider chmurowy

Venice może przejąć planowanie po ustawieniu `LLM_PRIMARY=venice / cloud` oraz obsługiwać wyszukiwanie. Obowiązują go te same lokalne kontrakty i allowlista.

8.3. Qwen 2.5 14B — lokalny fallback

Model:

```
qwen2.5-14b-instruct-1m-abliterated
```

Odpowiada za awaryjne planowanie, gdy provider główny nie może odpowiedzieć, oraz za lokalny `behavior digest` aktywności Screenpipe: streszczenie, temat, intencję, decyzje i kontekst osób. Nie powinien być mylony z modelem embeddingowym.

8.4. Nomic Embed — lokalny model wektorowy

Model:

```
text-embedding-nomic-embed-text-v2-moe
```

Odpowiada za:

- zamianę tekstu aktywności Screenpipe na wektory,
- zamianę pytania użytkownika na wektor zapytania,
- umożliwienie wyszukiwania semantycznego.

Nie odpowiada za rozmowę ani planowanie. Zwracany wektor ma obecnie 768 wymiarów.

9. Screenpipe i pamięć kontekstowa

9.1. Co rejestruje Screenpipe

Skrypt `scripts/start-screenpipe.ps1` uruchamia Screenpipe z:

- wszystkimi monitorami,
- wybranym fizycznym mikrofonem,
- fizycznymi wyjściami audio,
- zdarzeniami klawiatury,
- zdarzeniami schowka,
- zdarzeniami przewijania,
- retencją 14 dni,
- telemetrią wyłączoną,
- lokalnym API chronionym tokenem.

Globalna transkrypcja Screenpipe jest wyłączona.

9.2. Ważna uwaga prywatności

Konfiguracja Screenpipe jest szeroka:

- `ignore-incognito-windows=false` ,
- `use-pii-removal=false` ,
- `capture-on-keystroke=true` ,
- `capture-on-clipboard=true` .

To oznacza, że lokalny magazyn Screenpipe może zawierać bardzo wrażliwe dane. Nie wolno kopiować katalogu `%USERPROFILE%\screenpipe` do repozytorium ani udostępnić go bez świadomej decyzji użytkownika.

9.3. Vector memory worker

`ScreenpipeVectorMemoryWorker` :

1. uruchamia się razem z FastAPI,
2. odczytuje ostatnią aktywność tekstową i OCR Screenpipe,
3. lokalny Qwen tworzy ustrukturyzowany digest zachowania,
4. Nomic tworzy pięć oddzielnych embeddingów,
5. worker zapisuje named vectors oraz payload w Qdrant,
6. przy `QDRANT_DUAL_WRITE=true` zapisuje także wektor `semantic` w SQLite,

7. przy indeksowaniu wiąże nowy materiał z podobną wcześniejszą historią.

Worker nie zapisuje zrzutów obrazu w Qdrant. Zapisuje tekst/digest, metadane oraz embeddingi. Gdy Qdrant lub digester nie są skonfigurowane, używa starszej ścieżki metadanych i SQLite.

9.4. Selektywna transkrypcja spotkań

`ScreenpipeMeetingTranscriber` :

- wykrywa zakończone spotkania,
- czeka na grace period,
- sprawdza kontekst aplikacji i domen,
- blokuje YouTube bez wyjątków,
- przesyła do Deepgram wyłącznie pasujące fragmenty audio spotkań,
- zapisuje transkrypcje do SQLite,
- nie uruchamia globalnej transkrypcji każdego dźwięku.

Zapisane transkrypcje są grupowane per spotkanie, analizowane lokalnie przez Qwen i dodawane do Qdrant jako `screenpipe_meeting`.

10. Pamięć: SQLite i Qdrant

Plik:

```
data\voiceloop.db
```

SQLite działa w trybie WAL z `busy_timeout=10000`.

10.1. Tabele

Tabela	Przeznaczenie
commands	request, status, plan, provider, model, wyniki, błędy
conversation	historia wiadomości użytkownika i asystenta
memories	jawne fakty i preferencje
app_state	trwały stan workerów i znaczniki czasu
screenpipe_meeting_jobs	stan obsłużonych spotkań
screenpipe_transcripts	selektywne transkrypcje Deepgram
vector_memories	tekst, metadata i embeddingi Screenpipe

10.2. Qdrant — główny retrieval

Kolekcja `voiceloop_memory` używa pięciu named vectors i cosine distance. Payload zawiera m.in. źródło, stabilny `source_id`, typ pamięci, treść digestu, metadata oraz opcjonalne identyfikatory osoby, wizyty lub spotkania.

Qdrant działa lokalnie w trwałym kontenerze Docker. Pierwszy zapis tworzy kolekcję i indeksy payloadu. Migrator kopiuje istniejące rekordy SQLite bez usuwania starej bazy.

10.3. SQLite `vector_memories`

Rekord zawiera:

- źródło,
- stabilny `source_id`,
- tytuł,
- treść,
- metadata JSON,
- embedding JSON,
- wymiar wektora,
- czas utworzenia i aktualizacji.

Wektory są zapisane jako JSON, nie jako natywny typ wektorowy SQLite. Tabela pełni rolę dual-write, źródła migracji i fallbacku, gdy Qdrant nie zwróci wyników. Fallback SQLite nadal wyszukuje liniowo w ograniczonej puli rekordów.

11. Dozwolone akcje

action_id	Ryzyko	Potwierdzenie	Funkcja
open_calendar	low	nie	otwiera kalendarz Windows
open_browser	low	nie	otwiera pustą kartę przeglądarki
open_url	low	nie	otwiera poprawny URL HTTP/HTTPS
open_chat	low	nie	otwiera ChatGPT
describe_active_window	low	nie	odczytuje tytuł i proces aktywnego okna
minimize_active_window	low	nie	minimalizuje aktywne okno
minimize_all_windows	low	nie	minimalizuje wszystkie okna / pokazuje pulpit
describe_recent_activity	low	nie	odczytuje metadane aktywności Screenpipe
create_note	medium	nie	tworzy notatkę przez ustalone makro
run_uivision_macro	medium	tak	uruchamia istniejące makro z allowlisty
remember	medium	tak	zapisuje fakt lub preferencję
recall	low	nie	tekstowo filtruje jawną pamięć
speak_text	low	nie	uruchamia lokalny TTS Windows

recall przeszukuje obecnie tabelę memories tekstowo. Semantyczny retrieval vector_memories jest wykonywany automatycznie przed planowaniem i nie ma jeszcze osobnego publicznego action_id.

12. API VoiceLoop

12.1. Endpointy

Metoda	Ścieżka	Autoryzacja	Cel
GET	/	lokalna sesja	panel WWW
GET	/api/v1/session	tylko loopback	token i aktywny tryb LLM dla panelu
GET	/api/v1/health	brak	status komponentów
POST	/api/v1/commands	token	utworzenie komendy
GET	/api/v1/commands	token	lista komend
GET	/api/v1/commands/{id}	token	stan komendy
POST	/api/v1/commands/{id}/confirm	token	potwierdzenie
POST	/api/v1/commands/{id}/cancel	token	anulowanie
POST	/api/v1/stop	token	soft barge-in: ucina TTS i wraca do słuchania
POST	/api/v1/conversation/interrupt	token	soft barge-in rozmowy
POST	/api/v1/conversation/start	token	jedno powitanie TTS + Deepgram one-shot
POST	/api/v1/conversation/resume	token	natychmiast kończy aktywną przerwę czasową
POST	/api/v1/conversation/stop	token	twarde zakończenie sesji rozmowy
POST	/api/v1/listening/start	token	start Deepgram live
POST	/api/v1/listening/once?mode=...	token	jedna wypowiedź: assistant, note lub remember
POST	/api/v1/listening/stop	token	stop Deepgram live
GET	/api/v1/memories	token	jawna pamięć
POST	/api/v1/memories	token	dodanie pamięci
DELETE	/api/v1/memories/{id}	token	usunięcie pamięci
GET	/api/v1/events	brak	lokalny stream SSE

OpenAPI:

<http://127.0.0.1:8765/api/docs>

12.2. Token lokalny

Token jest generowany przy pierwszym starcie:

```
data\voiceloop.token
```

Mutujące i wrażliwe endpointy wymagają:

```
X-VoiceLoop-Token: <token>
```

`/api/v1/session` wydaje token tylko klientowi z loopback. SSE nie ma osobnego sprawdzenia tokenu, dlatego port `8765` nie może być wystawiony do sieci.

`N8nClient` może wysłać ten token do webhooka, ale aktualny workflow n8n nie weryfikuje nagłówka. Ochroną webhooka n8n jest obecnie wyłącznie bind loopback.

13. Statusy komend

Cykl życia:

```
received
  → planning
  → awaiting_confirmation
  → queued
  → executing
  → succeeded
```

Ścieżki końcowe:

```
failed | cancelled | rejected
```

Znaczenie:

- `received` — zapisano wejście,
- `planning` — trwa routing lub planowanie,
- `awaiting_confirmation` — potrzebna zgoda albo doprecyzowanie,
- `queued` — plan oczekuje,
- `executing` — akcje są wykonywane,
- `succeeded` — wszystkie wymagane kroki się udały,
- `failed` — planer lub akcja zwróciły błąd,
- `cancelled` — anulowano ręcznie lub przez STOP,
- `rejected` — np. pełna kolejka.

14. Panel WWW

Plik:

```
panel\index.html
```

Funkcje:

- pole rozmowy,
- wysyłanie komend,
- podgląd planu i odpowiedzi,
- opcjonalne `include_screen`,
- informację o aktywnym trybie LLM,
- checkbox chmurowego fallbacku tylko dla trybu local-first,
- start i stop Deepgram,
- STOP,
- lista zadań i statusów,
- potwierdzanie oraz anulowanie,
- lista jawnych wspomnień,
- SSE do aktualizacji w czasie rzeczywistym.

Przy `LLM_PRIMARY=cloud` lub `venice` checkbox jest wyłączony, a panel jawnie pokazuje, że Venice jest modelem głównym ustawionym po stronie serwera.

Stary adres `panel\deepgram.html` nie zawiera już klienta Deepgram ani pola klucza. Wyświetla komunikat migracyjny i przekierowuje do głównego panelu.

15. Deepgram

15.1. Nasłuch mikrofonu

`DeepgramListener`:

- otwiera WebSocket do modelu z `DEEPGRAM_MODEL` (domyślnie `nova-3`),
- używa języka z `DEEPGRAM_LANGUAGE` (domyślnie `p1`),
- przesyła mono PCM 16 kHz,
- publikuje interim i final transcripts wraz z `speaker_ids`,
- agreguje finalne fragmenty wypowiedzi,

- przetwarza zwykłe callbacki w ograniczonych, sekwencyjnych kolejkach,
- kieruje STOP, pauzę, potwierdzenie i wznowienie osobną ścieżką priorytetową, która nie czeka za odpowiedzią modelu i nie jest odrzucana przy przepełnieniu,
- ma keepalive,
- ponawia połączenie z backoffem,
- serializuje start/stop blokadą lifecycle, aby równoległe `start_once` nie tworzyły osieroconych strumieni mikrofonu.

Live STT domyślnie dodaje `diarize_model=latest` (obecnie streaming diarizer v1). Parametr `diarize=true` jest przestarzały i nie jest wysyłany.

15.2. Diaryzacja i ochrona przed rozmową w tle

Diaryzacja Deepgram rozróżnia zmiany mówców przy małym koszcie opóźnienia, ale zwraca wyłącznie lokalne numery `speaker` — nie rozpoznaje właściciela głosu. Ponieważ half-duplex zestawia nowy WebSocket po TTS, identyfikatory nie są trwałe między turami.

Zastosowane zabezpieczenia:

- co najmniej dwa numery mówców w jednej wypowiedzi → ignorowanie bez zwrotu do modelu, chyba że tekst zaczyna się od `Asystencie...` / `Venice...`,
- po `CONVERSATION_DIRECT_ADDRESS_AFTER_SECONDS` (domyślnie 30 s) kończy się okno swobodnego follow-upu i ponownie jest wymagany bezpośredni zwrot,
- podczas potwierdzonej przerwy przechodzą wyłącznie frazy wznowienia,
- echo aktualnego TTS jest porównywane z interim/final i nie uruchamia barge-in; filtr obejmuje krótkie asymetryczne fragmenty 2–3 słów oraz końcówkę odebraną do 2,5 s po zakończeniu TTS, a bezpośredni zwrot `Asystencie...` omija ten filtr,
- po `CONVERSATION_BARGE_IN_AFTER_MS` dokładne STOP działa już na interim, a pozostała mowa czeka na final Deepgram, aby sam szum lub niestabilny interim nie uciął rozpoczętego zdania; VoiceAttack/API omijają okres ochronny.

To jest filtr rozmowy w tle, nie speaker verification. Pewna identyfikacja konkretnej osoby wymaga enrollmentu i osobnego modelu embeddingów głosu.

15.3. Spotkania Screenpipe

Oddzielny worker używa Deepgram tylko dla zakończonych spotkań spełniających lokalną politykę. Nie należy łączyć tej funkcji z globalnym nagrywaniem audio.

15.4. Sekret

Klucz Deepgram należy przechowywać tylko w:

```
listener\.env
```

Historycznie klucz występował w starym panelu, dlatego jego rotacja jest zalecana, jeśli nie została wykonana.

16. n8n

Pliki:

```
n8n\voice-loop.json
scripts\start-n8n.bat
```

Założenia:

- wersja bazowa: n8n 2.33.7,
- uruchomienie przez npm,
- bind wyłącznie `127.0.0.1`,
- `Execute Command` wyłączone,
- n8n nie wykonuje dowolnego shell,
- n8n zwraca tylko typowany intent/action.

Aktualny workflow jest mały i celowo deterministyczny. Nie jest jeszcze rozbudowanym orkiestratorem integracji zewnętrznych.

Aktualny workflow nie sprawdza `X-VoiceLoop-Token`, mimo że klient może wysłać ten nagłówek. Nie wolno wystawiać portu `5678` do sieci przed dodaniem jawnej weryfikacji sekretu webhooka.

17. UI.Vision

Źródło prawdy makr:

```
uivision\macros\
```

Runtime:

```
%USERPROFILE%\Desktop\uivision
```

Skrypty:

- `sync-uivision.ps1` — synchronizacja makr,
- `run-uivision.ps1` — walidowany runner,
- `voiceloop_notatka.json` — tworzenie notatki.

Runner:

- dopuszcza wyłącznie bazową nazwę pliku `.json`,
- odrzuca separatory katalogów i segmenty `..`,
- sprawdza, że rozwiązana ścieżka pozostaje w katalogu runtime `macros`,
- wymaga makra na allowliście repozytorium i jego zsynchronizowanej kopii runtime,
- koduje argumenty URL,
- uruchamia osobne okno Chrome,
- zapisuje osobny log,
- wykrywa błąd,
- kończy po timeout,
- może zostać zatrzymany przez STOP.

18. VoiceAttack

VoiceAttack jest warstwą szybkich, jawnych komend głosowych i pewnym wyzwaczem swobodnego asystenta. Profil `VoiceLoop v2 PR0` ma 622 naturalne warianty 30 komend, ale nie używa wildcardu VoiceAttack do dyktowania długich zdań. Po stałej frazie „Asystent” otwiera na jedną wypowiedź polski STT Deepgram.

Pliki:

```
voiceattack\VoiceLoop-v2.vap
voiceattack\INSTRUKCJA.md
scripts\build-voiceattack-profile.py
scripts\va\*.vbs
scripts\send-command.ps1
```

Profil zawiera 30 komend:

- jednorazowy asystent: `Asystent`, `Hej asystent`,
- przechwycenie treści: `Zapisz notatkę`, `Zapamiętaj`,
- decyzje: `Potwierdź`, `Anuluj zadanie`,
- kontekst lokalny: `Co robiłem ostatnio`, `Opisz aktywne okno`, `Sprawdź pole tekstowe`,
- okna: `Zminimalizuj okno`, `Zminimalizuj wszystkie` / `Pokaż pulpit`, potwierdzone `WM_CLOSE` dla okna pod kursorem,
- kursor i schowek: kopiowanie tekstu, zaznaczenia, e-maila, numeru i zdania oraz zaznaczanie zdania lub akapitu przez UI Automation,
- Deepgram: `Włącz nasłuch`, `Wyłącz nasłuch`,
- diagnostyka: `Status Voice Loop`, `Test pętli`,
- akcje: kalendarz, przeglądarka, wyszukiwanie, zapamiętanie ostatniego źródła, czat, GPT i Gemini,
- panic button: `Stop teraz`, `Przerwij wszystko`.

18.1. Przebieg „Asystent”

„Asystent”

- VoiceAttack uruchamia `assistant.vbs`
- `send-command.ps1`
- `POST /api/v1/listening/once?mode=assistant`
- lokalny TTS: „Słucham”
- Deepgram odbiera jedną wypowiedź
- po `speech_final` zamyka połączenie
- `CommandRequest(source=deepgram)`
- `n8n` → `deterministic router` → `vector retrieval` → `Venice/Qwen`
- `executor` i odpowiedź głosowa

Tryb ma timeout 30 sekund. `mode=note` poprzedza transkrypcję tekstem „Zapisz notatkę”, dzięki czemu lokalny router buduje `create_note`. `mode=remember` dodaje „Zapamiętaj” i tworzy akcję `remember`, która zgodnie z polityką wymaga osobnego potwierdzenia.

18.2. Dispatcher

Wszystkie wrappery VBS uruchamiają `scripts\send-command.ps1`. Dispatcher:

- odczytuje lokalny token,
- obsługuje zwykłe `command_id` i tekst,
- przy aktywnym Deepgramie mapuje komendy VA na naturalny tekst,
- uruchamia tryb one-shot lub ciągły Deepgram,
- czyta krótki status usług,
- potwierdza albo anuluje najnowszą oczekującą akcję,
- nie interpoluje transkrypcji jako kodu shell.

STOP używa dedykowanego `/api/v1/stop` i nie czeka na `n8n` ani model. Potwierdzenie zachowuje się poprawnie po restarcie rdzenia, ponieważ `executor` może odtworzyć plan z SQLite, ale zgoda wygasa po 5 minutach. Dispatcher również ignoruje starsze wpisy `awaiting_confirmation`. Plik profilu jest generowany deterministycznie z istniejącego eksportu XML. Generator wykrywa kolizje fraz i brakujące wrappery VBS. Po przeniesieniu repozytorium należy ponownie uruchomić `scripts\build-voiceattack-profile.py`, ponieważ akcje VoiceAttack zawierają bezwzględne ścieżki.

19. TTS

`tts.py` używa lokalnych mechanizmów Windows. TTS:

- wypowiada odpowiedź dla źródeł Deepgram i VoiceAttack,
- dla akcji odczytujących dane wypowiada końcowy `ActionResult`, a nie tylko komunikat „Sprawdzam”,
- po potwierdzeniu `remember` wypowiada wynik „Zapamiętano”,

- zwykłe odpowiedzi planu działają jako śledzone taski asyncio,
- odczyt końcowego wyniku jest sekwencjonowany na końcu wykonania planu,
- może zostać przerwany przez STOP,
- nie jest przerywany przez własne echo: barge-in odrzuca transkrypt podobny do aktualnie czytanego tekstu, w tym krótkie fragmenty i końcówkę z bufora Deepgram odebraną tuż po ponownym otwarciu mikrofonu,
- oblicza timeout Azure SDK i fallbacku REST z długości pełnego tekstu oraz tempa głosu zamiast ucinąć odczyt po stałych 20 sekundach,
- publikuje `conversation.tts_completed` z flagami `completed` i `interrupted`.

Przed TTS odpowiedź rozmowy przechodzi kontrolę protokołu. `finish_reason=length`, niekończący status API, artefakt JSON albo brak `.`, `?`, `!` lub `...` na końcu powodują ponowne wygenerowanie całej odpowiedzi. Po dwóch nieudanych próbach tekst nie jest czytany jako pozornie poprawna odpowiedź.

Plan ma wewnętrzną flagę `speak_result`. `AssistantService` ustawia ją wyłącznie dla żądań głosowych i wybranych akcji (`create_note`, `remember`, `recall`, `describe_active_window`, `describe_recent_activity`). Executor wypowiada wtedy komunikat zwrócony przez akcję. Status VoiceAttack używa krótkiego lokalnego TTS w dispatcherze PowerShell, ponieważ nie tworzy wpisu w kolejce komend.

20. Konfiguracja

Wzór:

```
listener\.env.example
```

Właściwy plik lokalny:

```
listener\.env
```

Najważniejsze grupy ustawień:

Rdzeń

```
VOICELoop_HOST=127.0.0.1
VOICELoop_PORT=8765
VOICELoop_DATA_DIR=../data
COMMAND_DEDUPE_SECONDS=2
COMMAND_QUEUE_LIMIT=10
```

Gemini jako primary

```
LLM_PRIMARY=gemini
GEMINI_BASE_URL=https://generativelanguage.googleapis.com/v1beta/openai/
GEMINI_API_KEY=
GEMINI_MODEL=gemini-3.5-flash
```

Venice jako opcjonalny provider

```
CLOUD_LLM_ENABLED=false
CLOUD_LLM_BASE_URL=https://api.venice.ai/api/v1
CLOUD_LLM_API_KEY=
CLOUD_LLM_MODEL=venice-uncensored-1-2
```

Qwen jako fallback

```
LM_STUDIO_BASE_URL=http://127.0.0.1:1234/v1
LM_STUDIO_MODEL=qwen2.5-14b-instruct-1m-abliterated
```

Embeddingi

```
LOCAL_EMBEDDINGS_ENABLED=true
LOCAL_EMBEDDINGS_MODEL=text-embedding-nomic-embed-text-v2-moe
LOCAL_EMBEDDINGS_TIMEOUT_SECONDS=30
VECTOR_MEMORY_CONTEXT_LIMIT=8
```

Screenpipe

```
SCREENPIPE_ENABLED=true
SCREENPIPE_BASE_URL=http://127.0.0.1:3030
SCREENPIPE_LOOKBACK_DAYS=14
SCREENPIPE_VECTOR_MEMORY_ENABLED=true
SCREENPIPE_VECTOR_POLL_SECONDS=120
SCREENPIPE_VECTOR_RECENT_MINUTES=60
```


Deepgram

```
DEEPGRAM_API_KEY=  
DEEPGRAM_MODEL=nova-3  
DEEPGRAM_LANGUAGE=pl  
DEEPGRAM_DIARIZATION_ENABLED=true  
DEEPGRAM_DIARIZATION_MODEL=latest  
SAMPLE_RATE=16000  
AUTO_START_LISTENING=false  
CONVERSATION_DIRECT_ADDRESS_AFTER_SECONDS=30  
CONVERSATION_IGNORE_MULTI_SPEAKER=true  
# MICROPHONE_DEVICE=Microphone Array
```

Nigdy nie kopiuj wartości z `listener\.env` do dokumentacji, logów, commitów ani wiadomości.

21. Uruchamianie

21.1. LM Studio

W LM Studio:

1. uruchom Local Server na `127.0.0.1:1234`,
2. załaduj:
 - `text-embedding-nomic-embed-text-v2-moe`,
 - `qwen2.5-14b-instruct-1m-abliterated`,
3. upewnij się, że oba modele mają status READY.

21.2. Cały stack

```
cd C:\Users\marci\VoiceLoop  
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\start-all.ps1
```

Opcjonalny limit oczekiwania:

```
.\scripts\start-all.ps1 -TimeoutSeconds 600
```

Skrypt:

- ostrzega, jeśli LM Studio nie działa,
- synchronizuje wersjonowane makra UI.Vision do runtime,

- uruchamia Qdrant,
- uruchamia Screenpipe,
- uruchamia n8n,
- uruchamia core,
- czeka na porty do 300 sekund domyślnie,
- otwiera panel.

Nie istnieje też jeden globalny skrypt resetujący wszystkie procesy i dane; opcja `-Restart` dotyczy wyłącznie Screenpipe.

21.3. Osobno

```
.\scripts\start-core.bat
.\scripts\start-n8n.bat
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\start-screenpipe.ps1
```

22. Diagnostyka

Health

```
Invoke-RestMethod http://127.0.0.1:8765/api/v1/health |
  ConvertTo-Json -Depth 6
```

Porty

```
Get-NetTCPConnection -State Listen |
  Where-Object LocalPort -in 1234,3030,5678,5679,8765
```

Smoke test

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\test-loop.ps1
```

Smoke test nie używa mikrofonu i nie wykonuje akcji ekranowych.

Testy Python

```
cd listener
.\.venv\Scripts\python.exe -m ruff check voiceloop ..\tests
.\.venv\Scripts\python.exe -m pytest -c pyproject.toml ..\tests
```

Konfigurację pytest należy jawnie wskazać przez `-c pyproject.toml`, ponieważ testy leżą katalog wyżej niż plik konfiguracyjny.

Logi

```
logs\voiceloop-core.out.log
logs\voiceloop-core.err.log
logs\uivision_*.txt
```

23. Mapa modułów Python

Plik	Odpowiedzialność
app.py	składanie serwisów, lifespan, FastAPI, health, endpointy
settings.py	konfiguracja Pydantic z .env
models.py	kontrakty request, plan, status, memory, health
assistant.py	orkiestracja requestu i tworzenie planu
router.py	szybkie reguły deterministyczne
model_router.py	Venice/Qwen, structured output, fallback
embeddings.py	OpenAI-compatible embeddings w LM Studio
behavior_digest.py	lokalna analiza aktywności przez Qwen
qdrant_memory.py	pięć named vectors, zapis i ważony retrieval Qdrant
memory.py	SQLite, komendy, rozmowy, pamięci, wektory
screenpipe.py	read-only klient lokalnego API Screenpipe
screenpipe_memory.py	worker indeksujący aktywność wektorowo
screenpipe_deepgram.py	selektywna transkrypcja spotkań
screenpipe_audio_policy.py	reguły dopuszczenia audio
screen.py	aktywne okno, screenshot, UI Automation
n8n_client.py	webhook n8n i konwersja odpowiedzi na plan
actions.py	allowlista, ryzyko i implementacja narzędzi
executor.py	potwierdzenia, kolejka, zależności, STOP
deepgram.py	live STT z mikrofonu
tts.py	lokalna synteza mowy Windows
events.py	lokalny event bus i SSE

24. Struktura repozytorium

```
VoiceLoop\  
├─ README.md  
├─ docs\  
│   ├── VOICELOOP_ARCHITECTURE_HANDOFF.md  
│   └─ VOICELOOP_ARCHITECTURE_HANDOFF.pdf  
├─ listener\  
│   ├── .env                sekrety lokalne, bez commita  
│   ├── .env.example        wzór konfiguracji  
│   ├── pyproject.toml  
│   ├── requirements.lock  
│   ├── start-listener.bat  
│   └─ voiceloop\           rdzeń Python  
├─ panel\  
│   ├── index.html  
│   └─ deepgram.html        przekierowanie ze starego adresu  
├─ n8n\  
│   └─ voice-loop.json  
├─ scripts\  
│   ├── start-all.ps1  
│   ├── start-qdrant.ps1  
│   ├── start-screenpipe.ps1  
│   ├── start-n8n.bat  
│   ├── start-core.bat  
│   ├── test-loop.ps1  
│   ├── send-command.ps1  
│   ├── build-voiceattack-profile.py  
│   ├── run-uivision.ps1  
│   ├── sync-uivision.ps1  
│   └─ va\  
├─ uivision\  
│   └─ macros\  
├─ voiceattack\  
│   ├── VoiceLoop-v2.vap  
│   └─ INSTRUKCJA.md  
├─ tests\  
├─ data\                    baza, token, screenshots  
└─ logs\                    logi runtime
```

25. Model bezpieczeństwa

Główne zabezpieczenia

- wszystkie usługi lokalne bindują się do loopback,
- sekrety są w `.env`, nie w panelu,
- mutujące API wymaga tokenu,
- model zwraca structured output,
- kroki spoza allowlisty są ignorowane lub odrzucane,
- lokalny kod podnosi ryzyko, jeśli model je zaniży,
- akcje high-risk zawsze wymagają zgody,
- UI.Vision uruchamia tylko istniejące makra,
- URL musi być HTTP albo HTTPS,
- executor działa sekwencyjnie,
- STOP anuluje system centralnie,
- n8n ma wyłączony Execute Command.

Granice i świadome kompromisy

1. Gemini jest głównym modelem, więc wolny język trafia do API Google.
2. Historia rozmowy i wybrany kontekst vector memory trafiają do aktywnego providera chmurowego.
3. `include_screen=true` może wysłać obraz aktywnego okna do aktywnego providera.
4. Screenpipe przechowuje bardzo szeroki lokalny kontekst.
5. SSE nie ma osobnej autoryzacji i opiera się na loopback.
6. Webhook n8n nie weryfikuje obecnie wysyłanego tokenu i opiera się na loopback.
7. SQLite, token i screenshots nie są szyfrowane przez VoiceLoop na dysku.
8. `open_url` nie wymaga potwierdzenia, ale sprawdza protokół i poprawność URL.
9. Istniejące makro UI.Vision nadal jest zaufanym kodem automatyzacji; walidacja ścieżki nie analizuje bezpieczeństwa jego treści.
10. Model „uncensored/abliterated” nie zastępuje lokalnej polityki bezpieczeństwa.

26. Znane ograniczenia

Funkcjonalne

- n8n obsługuje obecnie mało reguł i nie jest pełnym katalogiem integracji,
- jawne `memories` i `vector_memories` mają oddzielne mechanizmy wyszukiwania,
- nie ma panelu do przeglądania i usuwania vector memories,

- swobodny tryb „Asystent” jest dialogiem dwustopniowym, a nie pojedynczą frazą `Asystent <polecenie>`,
- VoiceAttack i Deepgram korzystają z tego samego urządzenia wejściowego; stałe komendy należy wypowiadać bez uruchamiania trybu „Asystent”,
- light barge-in działa po okresie ochronnym i filtruje echo, ale nie jest pełnym akustycznym echo cancellation,
- diaryzacja Deepgram wykrywa numery mówców tylko w bieżącym strumieniu; nie identyfikuje biometrycznie właściciela i nie zachowuje ID między połączeniami,
- nie ma systemu wersjonowanych migracji SQLite.

Skalowalność

- Qdrant odpytuje każdą z pięciu przestrzeni osobno i łączy wyniki w Pythonie,
- SQLite fallback nadal przechowuje embedding jako JSON i wyszukuje liniowo,
- nie ma automatycznego pruning danych Qdrant ani `vector_memories`,
- pełny re-ranking i kontrola retencji wymagają dalszej pracy.

Operacyjne

- n8n uruchamiane przez npm może wymagać migracji po przyszłej aktualizacji,
- jeśli VoiceAttack pozostanie uruchamiany jako administrator, jego skrypty dziedziczą niepotrzebnie wyższe uprawnienia; zalecany jest zwykły tryb użytkownika,
- profile VoiceAttack zawierają bezwzględne ścieżki tej instalacji,
- nie ma pełnego supervision ani wspólnego restartu wszystkich usług,
- aktualizacje LM Studio mogą zmienić nazwy lub format identyfikatorów modeli,
- dokładne urządzenia audio Screenpipe są wybierane heurystycznie.

27. Rekomendowany backlog

Priorytet 0 — hardening lokalnego API

1. Weryfikować sekret bezpośrednio w workflow n8n.
2. Dodać autoryzację albo bezpieczny kanał sesyjny dla SSE.
3. Wzmocnić model wydawania tokenu przez `/api/v1/session`.
4. Ustawić restrykcyjne ACL dla bazy, tokenu, screenshotów i danych Screenpipe.
5. Rozważyć podpisy lub sumy kontrolne zatwierdzonych makr UI.Vision.

Priorytet 1 — prywatność i kontrola kontekstu

1. Dodać osobny przełącznik „Wyślij historię/pamięć do Venice”.

2. Oddzielić `include_screen` na:
 - same metadane,
 - UI Automation,
 - obraz.
3. Zredagować URL, tytuły i sekrety przed przekazaniem do Venice.
4. Dodać allowlistę aplikacji Screenpipe do vector memory.
5. Dodać pruning vector memories i retencję.

Priorytet 2 — lepszy RAG

1. Wektoryzować transkrypcje zakończonych spotkań.
2. Indeksować krótkie fragmenty OCR tylko z dozwolonych aplikacji.
3. Dodać filtrowanie po czasie, aplikacji i typie źródła.
4. Łączyć wyniki semantyczne z recency score.
5. Dodać endpoint diagnostyczny pokazujący retrieved chunks.
6. Przejść na `sqlite-vec` po przekroczeniu kilku–kilkudziesięciu tysięcy rekordów.

Priorytet 3 — UX

1. Dodać serwerowy tryb prywatności wyłączający wysyłanie kontekstu do Venice.
2. Pokazywać provider i model przy każdej odpowiedzi.
3. Dodać podgląd planu przed wykonaniem.
4. Dodać panel vector memories.
5. Rozważyć sprzętowe AEC lub push-to-talk jako uzupełnienie light barge-in.

Priorytet 4 — integracje

1. Rozbudować n8n o kalendarz, e-mail i przypomnienia.
2. Dodać typed adapters zamiast ogólnego workflow.
3. Dodać więcej testowanych makr UI.Vision.
4. Dodać tray app i autostart.

28. Jak bezpiecznie dodać nową akcję

1. Dodaj `ActionSpec` w `actions.py`.
2. Zdefiniuj ścisły `args_schema`.
3. Ustal minimalny uczciwy poziom ryzyka.
4. Ustaw `confirmation_required=True`, jeśli akcja:
 - wysyła,

- publikuje,
- usuwa,
- kupuje,
- loguje,
- zmienia konto,
- steruje cudzymi danymi.

5. Zaimplementuj handler bez interpretowania dowolnego shell.

6. Dodaj testy polityki.

7. Dodaj test powodzenia i błędu.

8. Uruchom ruff, pytest i smoke test.

9. Uzupełnij ten dokument.

29. Kryteria akceptacji zmian

Zmiana jest gotowa dopiero, gdy:

- nie wprowadza sekretu do repozytorium,
- zachowuje bind loopback,
- nie daje modelowi dowolnego shell,
- przechodzi `ruff`,
- przechodzi pełny `pytest`,
- przechodzi `scripts/test-loop.ps1`,
- healthcheck nie pogarsza istniejących komponentów,
- nowe akcje mają test ryzyka i potwierdzenia,
- dokumentacja odzwierciedla stan kodu.

30. Skrócona checklista dla następnego agenta

Przed pracą:

- ☐ przeczytaj ten dokument,
- ☐ sprawdź `git status`,
- ☐ nie czytaj wartości sekretów bez potrzeby,
- ☐ sprawdź `/api/v1/health`,
- ☐ sprawdź aktywne modele LM Studio,
- ☐ ustal, czy zmiana dotyczy Venice, Qwen czy Nomic.

Po pracy:

- ☐ ruff ,
- ☐ pełny pytest ,
- ☐ smoke test,
- ☐ healthcheck,
- ☐ brak sekretów w diffie,
- ☐ aktualizacja dokumentacji,
- ☐ brak niekontrolowanego procesu w tle.

31. Najważniejsza zasada architektoniczna

VoiceLoop ma oddzielać **rozumowanie** od **uprawnień**:

Venice/Qwen mogą proponować.

Tylko lokalny, walidowany kod może wykonywać.

To rozdzielenie jest ważniejsze niż wybór konkretnego modelu, dostawcy chmury, systemu RPA czy bazy wektorowej.